



Procedura *passo passo* per creare la tua versione personalizzata della piattaforma

Procedura passo passo, in 7 lingue, senza scrivere codice.

Una guida di Pessoa Academy · Vibe Coding Workflow™

10

PASSI

7

LINGUE GIÀ PRONTE

0

RIGHE DI CODICE
SCRITTE A MANO

~2 h

TEMPO MEDIO

Tu sei il *regista*, l'AI è il dattilografo

Tutta la guida si basa su una sola idea: tu spieghi cosa vuoi a parole, l'assistente AI scrive il codice nel tuo editor. Quando vedi una sezione PROMPT, copia la frase e incollala nel chat dell'assistente AI dentro Visual Studio Code.

Prepara la *cassetta degli attrezzi*

PERCHÉ Prima di scrivere qualunque cosa servono pochi strumenti, tutti gratuiti e ormai standard nel mondo del software. Non sono app a caso: ognuna ha un compito preciso e si aggancia alle altre. Pensa a un cantiere — **GitHub** è il magazzino online, **Visual Studio Code** l'officina dove apri i file, l'**assistente AI** il tuo apprendista che scrive sotto dettatura, **GitHub Desktop** il furgoncino che sposta materiale fra magazzino e officina, **Supabase** il deposito dei dati, **Netlify** il riflettore che mette tutto in vetrina sul web. Oggi è solo registrazione e installazione: niente codice.

COSA FAI

1. Vai su **github.com** e crea un account gratuito (basta un'email). GitHub è il diario dove ogni modifica al tuo codice viene salvata e datata: se sbagli, torni indietro al pomeriggio prima in un click. Da qui in poi è il punto fermo a cui tutto il resto si aggancia.
2. Scarica **Visual Studio Code** da code.visualstudio.com (Windows/Mac/Linux, ~100 MB). È il programma con cui apri, leggi e modifichi i file del progetto — pensa a Word ma per codice. Alla prima apertura sembra un editor di testo qualunque: la magia arriva al passo successivo, quando ci aggiungi l'AI.
3. In VS Code clicca l'icona *Extensions* nella barra laterale sinistra (un quadrato con quattro tasselli — scorciatoia Ctrl+Shift+X su Windows/Linux, Cmd+Shift+X su Mac). Cerca un assistente AI gratuito: il più semplice è **Gemini Code Assist** di Google, ma vanno bene anche **Codeium**, **Continue** o **Cody**. Installalo e fai login con un account Google: da questo momento dentro VS Code hai una chat a cui dare istruzioni a parole, in italiano.
4. Scarica **GitHub Desktop** da desktop.github.com e fai login con lo stesso account del primo passo. È un'app che fa da ponte fra il tuo computer e GitHub: con due click le tue modifiche locali finiscono online (commit + push). Senza questa app dovresti imparare i comandi da terminale di Git — ce la risparmiamo.
5. Vai su **supabase.com** e registrati (puoi usare Google o GitHub per fare prima). Supabase è il *magazzino online* dove l'app salverà tutto: utenti, partecipanti, mobilità, documenti caricati. Per ora ti basta avere l'account aperto — il progetto vero lo crei al passo 03, quando avremo bisogno delle sue credenziali.
6. Vai su **netlify.com** e registrati — il modo più rapido è "Sign up with GitHub", così i due servizi si parlano subito. Netlify è quello che trasformerà il tuo codice in un sito raggiungibile da chiunque sul web, gratuitamente. Niente server da configurare, niente FTP, niente di niente: ci penserà lui a ricompilare e ripubblicare ogni volta che salvi qualcosa su GitHub.
7. Scarica **Node.js** versione **LTS** (Long Term Support, la versione stabile) da nodejs.org. Node.js è il motore che fa girare il sito sul tuo computer mentre lo provi: senza, al passo 05 il comando `npm run dev` non saprebbe cosa eseguire. L'installazione è guidata, basta cliccare "Avanti" fino alla fine.

TRANQUILLO SUI COSTI Tutti questi strumenti sono gratuiti per uso personale: non viene chiesta nessuna carta di credito per iniziare. Se a un certo punto te la chiedessero — fermati e ricontrolla: stai per attivare un piano a pagamento per sbaglio, e per questa guida non serve mai.

Crea la tua *copia personale* del progetto

PERCHÉ "Forkare" significa fare una copia personale del progetto Managemob su GitHub: tu lavori sulla tua copia, l'originale resta intoccato. "Clonare" significa scaricare quella copia sul tuo computer per poterla aprire in VS Code. Sono due click e un comando: dopo questo passo hai il progetto vero, sul tuo disco, pronto a partire.

COSA FAI

1. Vai sulla pagina GitHub del template Managemob — es. github.com/ETN-International/managemob-app, oppure l'URL del template che ti è stato fornito. In alto a destra clicca **Fork**: GitHub ti chiede a quale account assegnare la copia (il tuo) e che nome darle. Scegli un nome breve in minuscolo, tipo *pessoa-mobility*. In pochi secondi la copia è tua.
2. Sulla pagina del tuo nuovo fork, clicca il bottone verde **<> Code** in alto a destra. Si apre un menu: scegli **Open with GitHub Desktop**. Il browser ti chiederà l'autorizzazione ad aprire l'app — accetta.
3. GitHub Desktop ti propone una cartella locale dove salvare il progetto. Lascia il default oppure scegli un percorso comodo (es. *Documenti* o *Desktop/Progetti*) e clicca **Clone**. In pochi secondi tutti i file del progetto sono sul tuo computer.
4. In alto a destra in GitHub Desktop clicca **Open in Visual Studio Code**: VS Code parte già con la cartella giusta caricata. Sulla sinistra vedi l'*Explorer* con tutta la struttura del progetto.
5. Apri la chat dell'assistente AI in VS Code e incolla questo prompt: Installa le dipendenze del progetto. L'AI scaricherà per te tutte le librerie su cui si appoggia il progetto (un paio di minuti). Quando ti dice di aver finito senza errori, sei pronto al passo successivo.

PROMPT DA INCOLLARE NELL'ASSISTENTE AI

"Spiegami in un paragrafo cosa contiene la cartella src/ — quali sono le sotto-cartelle principali e a cosa servono. Non modificare nulla per ora, voglio solo capire la mappa del progetto."

Prepara il *magazzino dei dati*

PERCHÉ Supabase è il "magazzino" online dove l'app salva tutto: utenti, partecipanti, mobilità, documenti. In questo passo crei lo spazio Supabase, ti procuri le due credenziali (URL + chiave) che servono al sito per parlarci, e crei il tuo primo utente amministratore. Tutto il resto — riempire il database con le tabelle giuste, blindare la sicurezza — lo farà l'assistente AI per te al passo successivo. Tu pensa solo ai click.

COSA FAI

1. Vai su supabase.com, fai login (o registrati se è la prima volta) e clicca **New project**. Scegli un nome (es. *pessoa-mobility*), una **regione in Europa** (Frankfurt, Ireland o Stockholm — più è vicina ai tuoi utenti, più velocemente l'app risponde) e una password robusta. **Salva la password** in un posto sicuro: non si recupera, ed eventualmente ti servirà più avanti.
2. Aspetta 1–2 minuti. Supabase sta preparando per te uno spazio dedicato. È normale che il pannello sia inattivo per qualche secondo — non si è bloccato.
3. Nella sidebar di sinistra clicca **Project Settings** → **API**. Trovi due valori scritti in chiaro: **Project URL** (un indirizzo che inizia con *https://*) e **anon public key** (una stringa lunga). Copiali entrambi e incollali in un blocco note temporaneo: ti serviranno fra un attimo, al passo 04. Niente di tecnico, sono due copia-e-incolla.
4. Sempre dalla sidebar, vai in **Authentication** → **Providers**. Verifica che **Email** sia attivo (di solito lo è di default). Per i test, disattiva *Confirm email*: così quando crei il tuo utente al passo successivo non devi confermare nulla via mail. Lo riattiverai prima di mettere il sito in produzione vera (vedi passo 10).
5. Sidebar → **Authentication** → **Users** → **Add user** → **Create new user**. Crea il tuo primo accesso da amministratore: una mail e una password che ricordi. È l'utente con cui ti loggherai al passo 05 per provare il sito.

Lascia che l'*assistente AI* colleghi tutto

PERCHÉ Ora il sito sa che esiste un magazzino Supabase, ma non ha ancora le credenziali per parlarci, e il magazzino è vuoto. In questo passo l'assistente AI fa per te tre cose tecniche in un colpo solo: salva le tue credenziali nel posto giusto, riempie il database con le tabelle dell'app, e attiva la sicurezza. Tu incolli un solo prompt e dici "sì" quando ti chiede conferma.

COSA FAI

1. Apri la chat dell'assistente AI dentro Visual Studio Code (in Gemini Code Assist è l'icona stellina nella barra laterale; in Cursor premi `Ctrl+L` / `Cmd+L`). Se vedi un'opzione tipo *Agent mode* o *@workspace*, attivala: significa che l'AI può vedere e modificare i file del progetto.
2. Copia tutto questo prompt (è già scritto, non devi capire ogni parola) e incollalo nella chat dell'AI:
3. Configurami il progetto Managemob con queste credenziali Supabase:
URL = `https://<tuo-progetto>.supabase.co`
KEY = `<la tua anon key>`

Per favore fai tre cose: (1) crea il file `.env` con queste due variabili (`VITE_SUPABASE_URL` e `VITE_SUPABASE_ANON_KEY`) e assicurati che `.env` sia in `.gitignore` così non finisce su GitHub; (2) esegui il file `supabase/schema.sql` sul mio database Supabase per creare le tabelle dell'app; (3) attiva la sicurezza Row Level Security su tutte le tabelle, così solo gli utenti loggati possono leggere e scrivere. Mostrami passo per passo cosa stai per fare e chiedi conferma prima di applicare.

4. Sostituisci `https://<tuo-progetto>.supabase.co` e `<la tua anon key>` con i due valori che hai copiato dal Supabase dashboard al passo 03. Premi Invio.
5. L'AI ti mostrerà ogni operazione una alla volta: "Sto creando il file `.env`, ok?", "Sto eseguendo questo SQL, ok?", "Sto attivando la sicurezza, ok?". A ogni domanda rispondi **sì** (o accetta il diff). Quando ti dice di aver finito, sei pronto: il sito è collegato a Supabase, il database ha le tabelle, la sicurezza è attiva — e tutto questo senza che tu abbia toccato un solo file a mano.

SE L'AI CHIEDE DI FARE QUALCOSA CHE NON CAPISCI Va bene dirgli "spiegami cosa stai facendo prima di farlo, in italiano semplice". È il suo lavoro spiegarti — non hai bisogno di studiare SQL o variabili d'ambiente per finire questa guida. Se ti sembra che stia facendo qualcosa di strano (es. cancellare file che non c'entrano), digli "fermati, non ho dato l'ok" e riprova con istruzioni più precise.

Vedi il sito girare in *locale*

PERCHÉ Prima di mettere il sito online, conviene provarlo sul tuo computer. È il momento in cui scopri se Supabase è collegato bene, se il login funziona, se i dati si salvano davvero. Se qualcosa non va lo vedi qui — senza fare danni a nessuno.

COSA FAI

1. In VS Code apri la chat dell'assistente AI e incolla: `Avvia il sito in locale per provarlo nel browser`. L'AI eseguirà il comando giusto (è `npm run dev`, se sei curioso). Se preferisci farlo a mano, apri il Terminale (*View* → *Terminal*), scrivi `npm run dev` e premi Invio.
2. Nel terminale apparirà una riga tipo `Local: http://localhost:XXXX`. Apri quell'indirizzo nel browser (di solito è `http://localhost:3000`; se la porta è occupata, Vite ne sceglie un'altra e te la mostra). Se l'AI ha avviato per te, te lo dice direttamente in chat.
3. Vedrai la schermata di login. Accedi con email e password dell'amministratore creato al passo 03. Se ti dice "Invalid login credentials", chiedi all'AI: `Il login non funziona, controlla le credenziali Supabase`. Spesso è un valore copiato male o uno spazio di troppo.
4. Una volta dentro, prova a creare un partecipante di prova dalla home (basta nome, cognome, paese di destinazione). Salva, poi premi F5 per ricaricare la pagina: se il partecipante è ancora lì, il database online sta salvando per davvero.
5. Clicca il selettore di lingua (in alto, sia nel login sia nella sidebar): l'interfaccia deve cambiare lingua all'istante in tutte e 7 le lingue. Se manca qualche traduzione o vedi qualcosa di strano, te ne accorgi qui.

SE QUALCOSA NON VA, PROMPT UTILE

"Se qualcosa non funziona quando avvio il sito, dimmi cosa c'è scritto nel terminale e proponi una correzione semplice. Non modificare file senza chiedermi prima."

Personalizza in 7 lingue

PERCHÉ Managemob parla già 7 lingue di default. Tu non devi tradurre da zero — devi solo riscrivere i testi visibili (etichette, titoli, frasi nei pulsanti) per adattarli alla TUA organizzazione, e in modo naturale in ogni lingua. Lo fa l'assistente AI con un solo prompt: tu scegli il tono, l'AI lo applica a tutti i 7 file in una volta sola.

COSA FAI

1. In VS Code apri la chat dell'assistente AI. Non devi aprire i 7 file delle lingue uno per uno — fa tutto lui.
2. Incolla questo prompt (sostituendo "Pessoa Academy" con il nome della tua organizzazione e descrivendo il tono che vuoi):
3. Riscrivi tutti i testi visibili dell'app per il brand 'Pessoa Academy', con un tono caldo e professionale (modifica qui se vuoi un altro stile). Apri tutti i 7 file in `src/locales/` (`it.ts`, `en.ts`, `es.ts`, `fr.ts`, `de.ts`, `pt.ts`, `se.ts`), modifica solo i valori (le stringhe fra virgolette), non toccare le chiavi (i nomi prima dei due punti). Traduci in modo naturale in ogni lingua, non parola per parola. Mostrami il diff prima di applicare.
4. L'AI ti propone tutte le modifiche in un'unica revisione. Scorri il diff: se ti piacciono le frasi, accetta. Se vuoi un tono diverso (più formale, più giovane, meno corporate), digli ad esempio "riscrivi più formale, dando del Lei" e te le rifa.

AGGIUNGERE UNA 8ª LINGUA Vuoi una lingua oltre alle 7 (es. polacco)? Chiedi all'AI: *"Aggiungi il polacco come ottava lingua dell'app, copiando da en.ts e traducendo. Aggiungi anche il selettore della lingua con la bandierina."* Lo fa lui in 30 secondi, senza che tu debba toccare codice.

Dai la *tua identità visiva*

PERCHÉ L'aspetto è la prima cosa che vedono gli utenti. Cambiando logo, colori e il titolo del sito, Managemob smette di chiamarsi Managemob e diventa visibilmente la TUA piattaforma. Sono modifiche puramente estetiche — la logica resta intatta — ma cambiano radicalmente la percezione.

COSA FAI

1. Prepara il tuo logo come file `logo.svg` (preferito, vettoriale che scala perfettamente) o `logo.png` (formato quadrato, almeno 256×256 pixel, sfondo trasparente). Trascina il file nella cartella `public/` del progetto in VS Code: vedrai apparire la nuova icona nell'Explorer.
2. Decidi i tuoi due colori: il **primario** (il colore principale dell'app, es. il blu della tua organizzazione) e l'**accento** (un colore complementare per dettagli e bottoni). Se non hai una palette pronta, vai su coolors.co, gioca finché ti piace, e annota i due codici esadecimali (es. `#1D72B8` e `#38BDF8`).
3. Apri la chat dell'AI e incolla questo prompt (sostituendo nome organizzazione e codici colore):
4. Rebrand completo dell'app per la mia organizzazione 'Pessoa Academy'. (1) Sostituisci il logo placeholder 'M' con il file `public/logo.svg` in `Sidebar.tsx` e `Login.tsx`. (2) Aggiorna i colori in `src/styles/main.css`: `--primary = #XXXXXX`, `--accent = #YYYYYY`. (3) Cambia il titolo del sito (tab del browser) in 'Pessoa Academy' in `index.html`, e fai puntare la favicon al mio nuovo logo. (4) Sostituisci la parola 'Managemob' con 'Pessoa Academy' nei testi visibili dei 7 file di `src/locales/`. Mostrami il diff prima di applicare ogni modifica.
5. L'AI propone tutte le modifiche. Approva. Ricarica il browser sulla pagina `localhost:3000`: vedi già la tua identità visiva applicata.

Automatizza i tuoi *documenti*

PERCHÉ Una delle cose più potenti di Managemob: prendi un documento Word qualunque (Learning Agreement, Europass Mobility, contratto di mobilità...) e lo trasformi in un template che si compila automaticamente con i dati del partecipante che selezioni. È la fine del copia-incolla manuale: 50 partecipanti, 50 documenti, 30 secondi.

COSA FAI

1. Apri il documento Word che vuoi automatizzare in Microsoft Word, LibreOffice o Google Docs. Se è un PDF o una scansione, prima dovrai ricostruirlo come **.docx** — l'app accetta solo file **.docx**.
2. Cerca i punti dove vanno i dati variabili (nome partecipante, date, azienda ospitante, paese...) e sostituiscili con tag fra parentesi graffe. Esempi pratici:
[Nome] [Cognome] → {name} {surname}
[Data inizio] → {arrival_date}
[Email] → {email}
[Azienda ospitante] → {host_company_name}. La lista completa dei tag disponibili è nella pagina *Documenti* dell'app, cliccando "Export Placeholders".
3. Salva in formato **.docx** (non .doc, non .pdf, non Google Docs). Da Word: *Salva con nome* → *Documento Word (*.docx)*.
4. Nell'app Managemob: *Documenti* → *Carica Template*. Dai un nome al template (es. "Learning Agreement IT"), seleziona il file .docx, conferma. Il template appare nella lista.
5. Selezionalo, cerca uno o più partecipanti dalla lista, clicca **Genera**. Per un solo partecipante scarichi un file singolo; per più partecipanti scarichi uno ZIP con un documento ciascuno. La formattazione (grassetto, corsivo, tabelle) viene preservata: l'app sostituisce solo i tag.

REGOLE IMPORTANTI DEI TAG I tag sono **case-sensitive**: scrivi {name}, non {Name}. Niente spazi dentro le graffe ({ name } non funziona). I tag inesistenti restano nel documento come testo letterale ("{nomeSbagliato}"): se il documento generato ha ancora delle graffe, hai sbagliato il nome del campo. Per la lista esatta vai su *Documenti* → *Export Placeholders (CSV)*.

Metti il sito *su Netlify*

PERCHÉ Per ora il sito gira solo sul tuo computer e nessuno lo vede. Per condividerlo con il mondo lo metti su Netlify, gratuito, collegato direttamente al tuo repository GitHub. Da quel momento, ogni volta che spingi modifiche su GitHub, Netlify ricostruisce il sito e lo ridistribuisce automaticamente — niente FTP, niente comandi di deploy, niente di niente.

A · Salva le modifiche su GitHub (con GitHub Desktop)

1. Apri GitHub Desktop. Se hai modificato qualche file (e dopo i passi 06–07 ne hai modificati parecchi), in alto a sinistra vedi la lista dei file cambiati con un'anteprima delle differenze in evidenza (verde = aggiunto, rosso = tolto).
2. In basso a sinistra c'è un campo per il messaggio di salvataggio: scrivi una descrizione breve di cosa hai fatto (es. "Personalizzazione Pessoa Academy: logo, colori, testi"). Clicca **Commit to main**: hai sigillato un punto di salvataggio.
3. In alto, clicca **Push origin**. Le tue modifiche sono ora sul tuo repository GitHub — finalmente backup-ate, finalmente pronte per Netlify.

B · Pubblica su Netlify (collegato al tuo repository)

1. Vai su netlify.com e fai login (il modo più rapido è *Sign up with GitHub*). Dashboard → **Add new site** → **Import an existing project** → **Deploy with GitHub**.
2. La prima volta Netlify ti chiede l'autorizzazione a leggere i tuoi repository — accettala. Poi seleziona dalla lista il **tuo fork** creato al passo 02.
3. Netlify riconosce automaticamente che è un progetto Vite e propone le impostazioni giuste. Verifica solo che ci sia: *Build command*: `npm run build` · *Publish directory*: `dist` · *Branch*: `main`.
4. Apri **Show advanced** → **Add environment variable**. Aggiungi le DUE variabili che l'AI ha messo nel tuo file `.env` al passo 04: `VITE_SUPABASE_URL` e `VITE_SUPABASE_ANON_KEY` (con gli stessi valori). Senza queste, l'app online non saprà come parlare con Supabase.
5. Clicca **Deploy**. Il primo build dura 1–3 minuti — vedi il log scorrere in tempo reale. Quando vedi "Site is live" hai finito.
6. Netlify ti dà un indirizzo random tipo `https://<parole-strane-123>.netlify.app`. **Quello è il tuo sito online!** Da *Site settings* → *Change site name* puoi rinominarlo (es. `pessoa-mobility.netlify.app`); da *Domain management* puoi collegare un tuo dominio se ne hai uno (es. `mobility.pessoa-academy.com`).

PROMPT DA INCOLLARE NELL'ASSISTENTE AI

"Aggiungi al progetto la configurazione che fa funzionare correttamente tutte le pagine interne quando il sito è online su Netlify (in modo che, se aggiorni una pagina come /dashboard, non veda l'errore 404 Not Found). Mostrami il file prima di crearlo."

Sei **online**, ultima verifica

PERCHÉ Sei online, ma prima di dare il sito agli utenti veri controlliamo che tutto funzioni dall'URL pubblico Netlify (non più dal localhost). Sono 7 controlli rapidi: se passano tutti, sei pronto. Se ne fallisce qualcuno, ora c'è ancora tempo per aggiustare senza che nessuno se ne accorga.

PRIMA DEL GO-LIVE UFFICIALE Quando passi dal beta interno agli utenti veri, fai questa pulizia in Supabase: (1) **riattiva Confirm email** in Authentication → Providers (così i nuovi iscritti devono confermare l'email); (2) **ruota la password DB** in Project Settings → Database (cambia la password che hai salvato al passo 03); (3) cancella l'**amministratore di test** e crea quelli reali; (4) fai un **backup** del database (Project Settings → Database → Backups). Da fare una volta, ti salva il sedere mille.

CHECKLIST FINALE

- Login dall'URL pubblico Netlify funziona (con l'amministratore creato al passo 03).
- Il selettore di lingua mostra correttamente tutte e 7 le lingue, e i testi cambiano davvero quando le clicco.
- Aggiungo un partecipante di prova, premo F5: se è ancora lì, il database online sta salvando.
- Genero un documento Word selezionando un partecipante: i tag {name} ecc. nel file scaricato sono sostituiti dai dati reali.
- Il Calendar mostra il partecipante alle date corrette.
- Aggiornare una sotto-pagina come /dashboard non dà errore 404 (= il prompt del passo 09 è stato applicato).
- Il logout torna alla schermata di login senza errori.

Una piattaforma **tua**, in 7 lingue, senza scrivere codice

Visual Studio Code apre i file. L'assistente AI scrive il codice. GitHub Desktop sincronizza online. Supabase conserva i dati. Netlify mette il sito in vetrina sul web. Tu coordini il tutto e prendi le decisioni che contano: nome, colori, testi, documenti automatici, lingue. È il Vibe Coding Workflow™: zero righe di codice scritte a mano, ma il risultato è una piattaforma tua al 100%.